UBER (help for zomato as well – quadtree)

Connects customers with drivers.
↳ Partner with Uber

Requirement (clear & crisp) → Design requirements & interact with interviewer

Customer

- ↳ Able to see nearby cabs.
- ↳ Able to request a ride.

Driver

- → Update current position (lat, long) / status
- → Notified whenever customer requests a ride.
- → Stop the trip

Once a trip is confirmed, notify both driver & customer.

Non-Functional Requirements

- scalable (for entire world)
- Real-time updation of driver coordinates
- Customer should've real-time updates about driver.

Estimation

No. of customers = 500 M
No. of drivers ≈ 2M

↳ Daily active drivers = 1M

↓

Total no. of daily rides = 2M

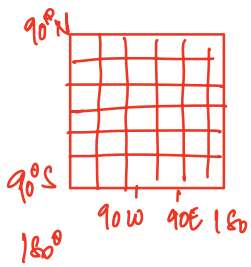
or As a user → able to find all drivers near a given position within a radius of x kms.

↑

Real-time :- get this info.

⇓

Soln : Break entire map into smaller grids (Each grid cell represents drivers etc corresponding to lat, long)



→ So, we need to see just neighbours : optimal search

Size of grid = radius

eg Radius = 10 km.

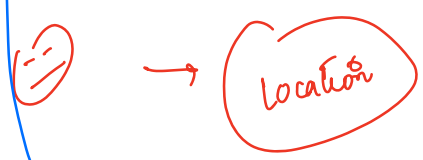
Area of grid = $10 \times 10\ km^2$

Total area of earth = 510 M km^2

Total no. of grids = $\frac{510\ M}{10 \times 10}$ = 5.1 Mn

Mapping

Location [lat, lng] = Grid ?



location ID
lat
long
→ correspond to [grid]

how many drivers?

↓

Hashmap data structure

Key: grid ID — Expedite the search

Value: list of drivers.

Each grid can be uniquely represented by 4 Bytes

∴ Total storage needed for grids = 5.1 Mn × 4 bytes

= 20.4 × 1000000 bytes

= 20.4 MB

= 0.02 GB

Estimate storage needed to store all locations

No. of locations : 500M

→ each locations → 8 byte

Total = 500,000,000 × 8

= 4000,000,000

= 4000 MB

= 4GB

Total size ⇒ 4 + 0.02 → 4.0024B

Challenges → Some grids — very dense, very sparse

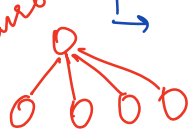
So, not all drivers get chance

Sol^n : Strategy - Dynamic size of grid → max^n : 100 drivers

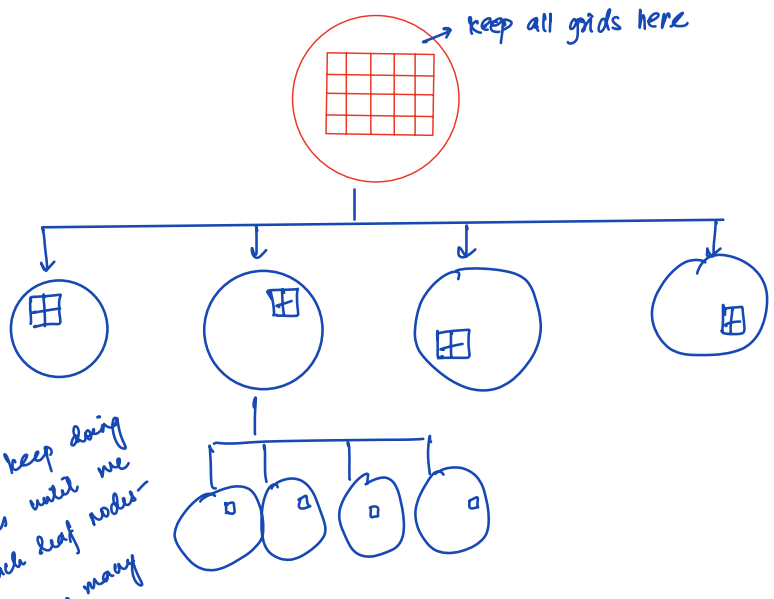
if 100 → then partition into multiple grids

min : 10 drivers → < then that - then, shrink

Can solve this using quad-tree datastructure



Start with a node – stick with it till < 100 [leaf Nodes will store all the drivers]



↳ else look for centers of this tree.

Storage Needed : locations : 500 M { location-id 8, lat 8, long 8 } 24 Bytes

∴ Total data = 500 M × 24 bytes

= 500 × 24 MB

= 12000 MB = 12GB

∴ Total grid = 5Mm } Total No. of leaf nodes → data stored by them is 12GB

stored as leaf node in quadtree

Data size because of tree structure

No. of leaf nodes = 5M

∴ Internal nodes = $1/3 \times 5M$

Each internal node → 4 children

Each pointer can be stored in 8 bytes

∴ Total size of the quadtree = $1/3 \times 5 \times 4 \times 8 = 2.05$ GB

∴ Total memory reqd = 12.05 GB

⇒ Each driver needs to update its location
↳ 5 seconds

⇓

Quadtree needs to be updated → very expensive & time-taking.

Sol^n : Don't update quad-tree very frequently eg say 2 (15 seconds)

↓

Impact : Customer'll not have near real-time address of the car ⇒ stale data.

⇓

Can maintain hashmap → store
- driver id : key. (2M → 2 bytes)
- prev-long (8 bytes)
- prev-lat (8 Bytes)
- current-lat (8 B)
- current-long (8B)

$\boxed{2M \times 35 \text{ Bytes} = 70 \text{ MB.}}$

↓

Fault Tolerance → Redundancy (Avoid single point of failure)

↓

$\boxed{\text{Driver's location map service}}$

↓

15 seconds interval

Quadtree updates

How to ensure customer gets latest updates?

① Users opens app

② Query to get list of drivers

③ Server'll make call to Quadrice → list of drivers

④ Server'll subscribe customer to the driver - So everytime driver updates its location on the map - it'll result in customer getting notified?

↓

How much storage? → 1M active drivers.
10 subs. to drivers.

Total subscribers = 10 Mn

↓

Info about driver → 1M × 3 bytes info
" " location → 1M × 10 × 8 (↑ subscribers)
} = ~~8~~ ≈ 3MB

How much bandwidth we'll need for broadcasting drivers location to customers?

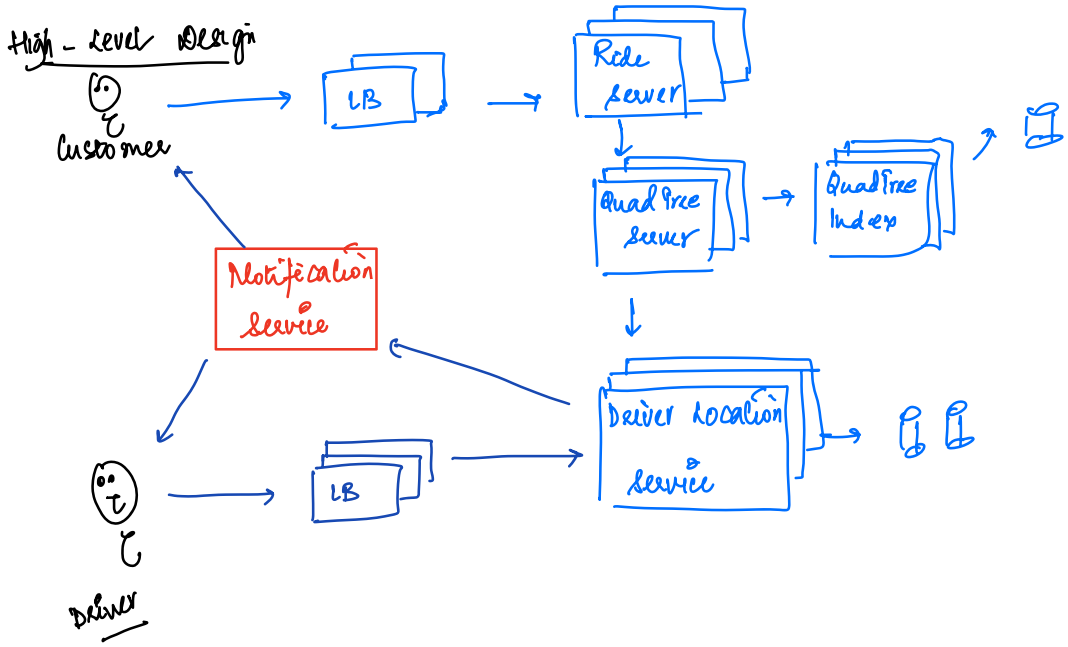
Each driver → 10 subscribers → Total: 10 Mn sub.

Data → Driver id = 3 Bytes
Location = 16 Bytes

∴ Total data = 10M × 16 = 160MB data to all customers

∴ Bandwidth ≈ 160 MB/s → Push notification

High-level design



How customer requests ride?

1. Opens app → requests for ride
2. Ride Server accepts request
   - talks to quadtree server – list of drivers
   - sort the driver IDs based on ratings
   - pick top 3 & notify them
3. As soon as driver accepts, notification is sent to both driver & customer.